Chapter 4

The Perceptron

4.1. The Basic Structure

The Perceptron, which is possibly the earliest neural computation model, is due to F. Rosenblatt and dates back to 1958 (see Sec. 3.1). We can consider the neuronal model using the signum nonlinearity, as in Sec. 3.4) to be a special case of the Perceptron. The Perceptron serves as a building block to most later models, including the Adaline discussed earlier whose neuronal model may be considered as a special case of the Perceptron. The Perceptron possesses the fundamental structure as in Fig. 4.1 of a neural cell, of several weighted input

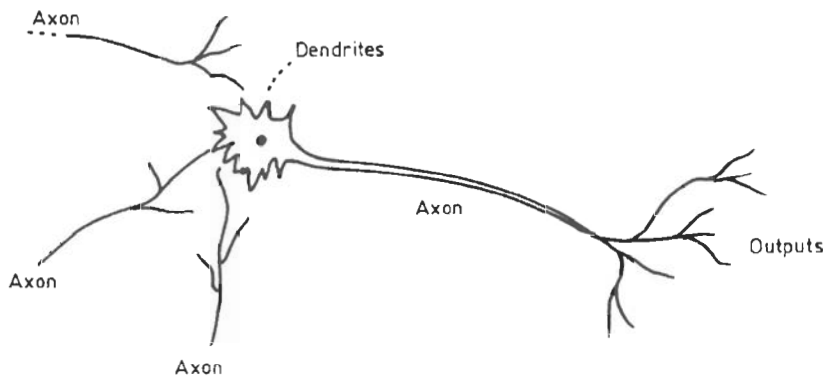


Fig. 4.1. A biological neuron.

connections which connect to the outputs, of several neurons on the input side and of a cell's output connecting to several other neural cells at the output side. It differs from the neuronal model of the Adaline (and Madaline) in its employment of a smooth activation function ("smooth switch" nonlinearity). However the "hard switch" activation function of the Adaline and of the Madaline may be considered as a limit-case of the Perceptron's activation function. The neuronal model of the unit of several weighted inputs/cell/outputs is the perceptron, and it resembles in structure, in its weighted inputs whose weights are adjustable and in its provision

2013. World Scientific.
All rights reserved. May not be reproduced in any form without permission from the publisher, except fair uses permitted under U.S. or applicable copyright law.
EBSCO Publishing: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD 622050; Daniel Graupe;
Principles Of Artificial Neural Networks (3rd Edition) Account:ehost.

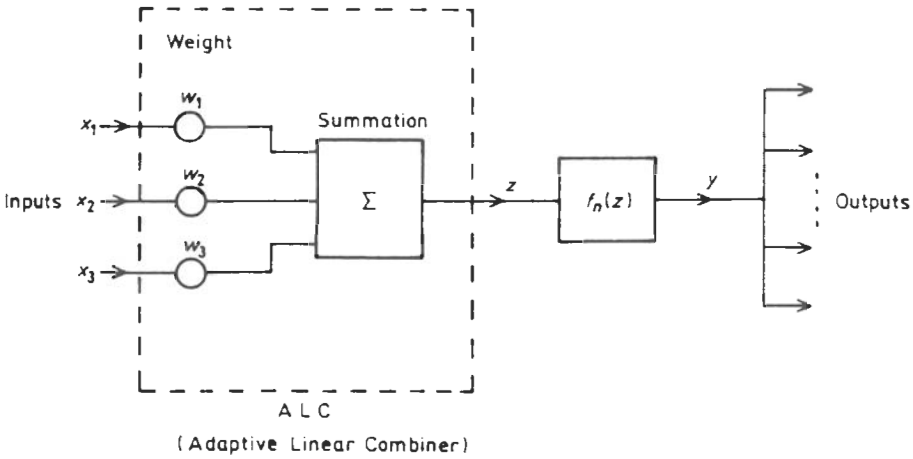


Fig. 4.2. A perceptron (artificial neuron).

for an output that is a function of the above weighted input, the biological neuron as in Fig. 4.2.

A network of such Perceptrons is thus termed a neural network of Perceptrons. Denoting the summation output of the ith Perceptron as z_i and its inputs as $x_{li} \cdots x_{ni}$, the Perceptron's summation relation is given by

$$z_i = \sum_{j=1}^{m} w_{ij} x_{ij} \tag{4.1}$$

w_{ij} being the weight (which are adjustable as shown below) of the jth input to the ith cell. Equation (4.1) can be written in vector form as:

$$z_i = \mathbf{w}_i^T \mathbf{x}_i \tag{4.2}$$

where

$$\mathbf{w}_i = [w_{i1} \cdots w_{in}]^T \tag{4.3}$$

$$\mathbf{x}_i = [x_{i1} \cdots x_{in}]^T \tag{4.4}$$

T being denoting the transpose of $\mathbf{w}$.

4.1.1. *Perceptron's activation functions*

The Perceptron's cell's output differs from the summation output of Eqs. (4.1) or (4.2) above by the activation operation of the cell's body, just as the output of the biological cell differs from the weighted sum of its input. The activation operation is in terms of an *activation function* $f(z_i)$, which is a nonlinear

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

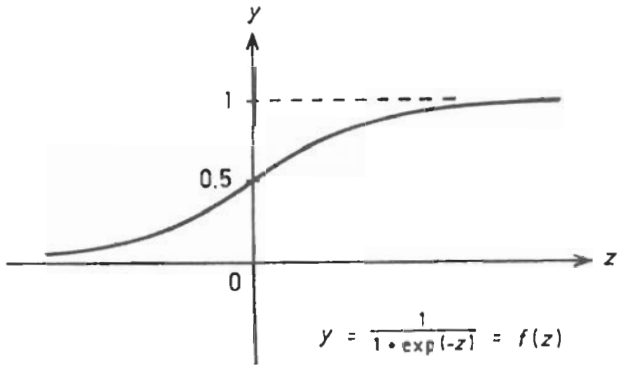


Fig. 4.3. A unipolar activation function for a perceptron.

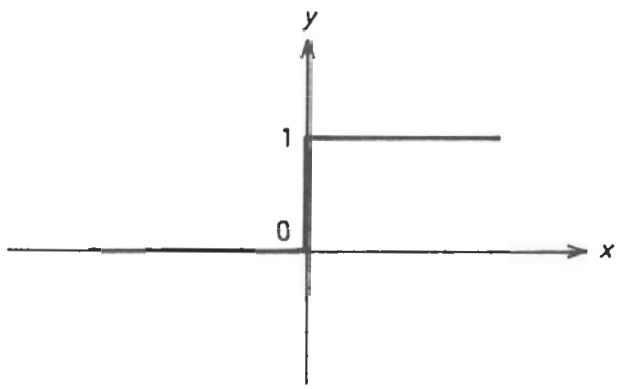


Fig. 4.4. A binary (0,1) activation function.

function yielding the ith cell's output y_i to satisfy

$$y_i = f(z_i) \tag{4.5}$$

The activation function f is also known as a squashing function. It keeps the cell's output between certain limits as is the case in the biological neuron. Different functions $f(z_i)$ are in use, all of which have the above limiting property. The most common activation function is the *sigmoid function* which is a continuously differentiable function that satisfies the relation (see Fig. 4.3), as follows:

$$y_i = \frac{1}{1 + \exp(-z_i)} = f(z_i) \tag{4.6}$$

such that for

$$\{z_i \to -\infty\} \Leftrightarrow \{y_i \to 0\};\ \{z_i = 0\} \Leftrightarrow \{y_i = 0.5\};\ \{z_i \to \infty\} \Leftrightarrow \{y_i \to 1\}$$

See Fig. 4.4.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

Another popular activation function is:

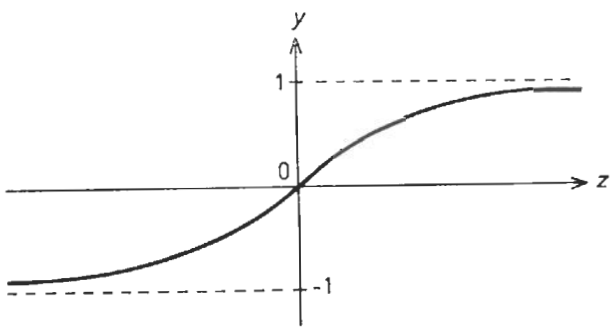
$$y_i = \frac{1 + \tanh(z_i)}{2} = f(z_i) = \frac{1}{1 - \exp(-2z_i)} \tag{4.7}$$

whose shape is rather similar to that of the S-shaped sigmoid function of Eq. (4.6), with $\{z_i \to -\infty\} \Leftrightarrow \{y_i \to 0\}$; $\{z_i = 0\} \Leftrightarrow \{y_i = 0.5\}$ and $\{z_i \to \infty\} \Leftrightarrow \{y_i \to 1\}$

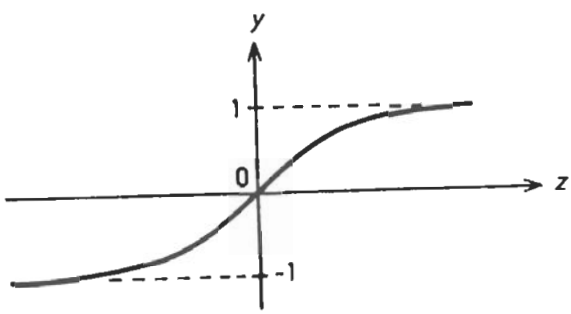
The simplest activation function is a hard-switch limits threshold element; such that:

$$y_i = \begin{cases} 1 & \text{for} \quad z_i \geq 0 \\ 0 & \text{for} \quad z_i < 0 \end{cases} \tag{4.8}$$

as in Fig. 4.4 and as used in the Adaline described earlier (Chap. 4 above). One may thus consider the activation functions of Eqs. (4.6) or (4.7) to be modified binary threshold elements as in Eq. (4.8) where transition when passing through the threshold is being smoothed.



(a) $y = \frac{2}{1+\exp(-z)} - 1$



(b) $y = \tan h(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

Fig. 4.5. Bipolar activation functions.

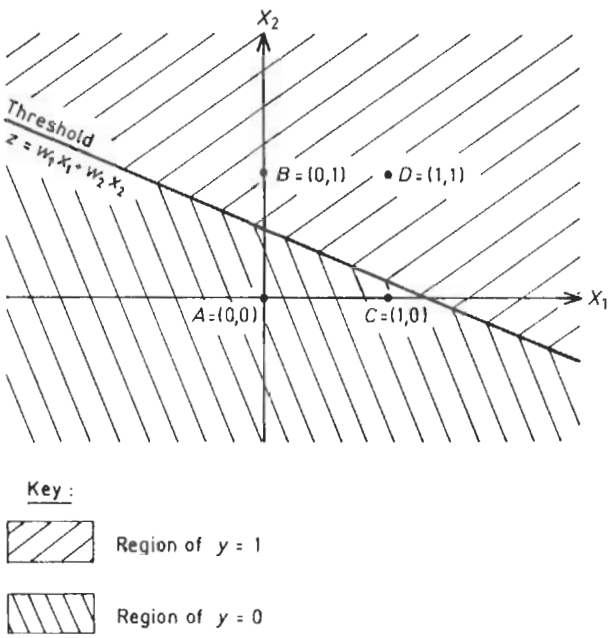
EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

In many applications the activation function is moved such that its output y: ranges is from -1 to $+1$ as in Fig. 4.5, rather than from 0 to 1. This is afforded by multiplying the earlier activation function of Eqs. (4.6) or (4.7) by 2 and then subtracting 1.0 from the result, namely, via Eq. (4.6):

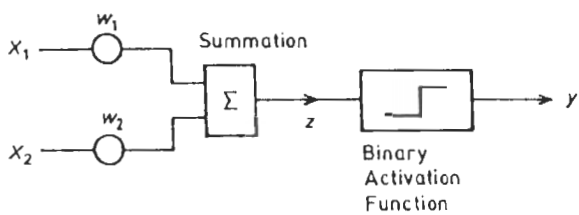
$$y_i = \frac{2}{1 + \exp(-z_i)} - 1 = \tanh(z_i/2) \tag{4.9}$$

or, via Eq. (4.7),

$$y_i = \tanh(z_i) = \frac{1 - \exp(-2z_i)}{1 + \exp(-2z_i)} \tag{4.10}$$



(a) Single-layer perceptron: 2-input representation



(b) Two-input perceptron

Fig. 4.6. Two-input perceptron and its representation.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

Although the Perceptron is only a single neuron (at best, a single-layer network), we present in Sec. 4.A below a case study of its ability to solve a simple linear parameter identification problem.

4.2. The Single-Layer Representation Problem

The *perceptron's learning theorem* was formulated by Rosenblatt in 1961. The theorem states that a perceptron can learn (solve) anything it can represent (simulate). However, we shall see that this theorem does not hold for a single Perceptron (or for any neuronal model with a binary or bipolar output, such as in Chap. 3) or for a single layer of such neuronal models. We shall see later that it does hold for models where the neurons are connected in a multi-layer network.

The single layer perceptron yields the representation description as in Fig. 4.6(a) for a two input situation. This representation holds for several such neurons in a single layer if they do not interconnect.

The above representation diagram results from the perceptron's schematic as in Fig. 4.6(b).

The representation of a 3-input perceptron thus becomes as in Fig. 4.7, where the threshold becomes a flat plane.

By the representation theorem, the perceptron can solve all problems that are or can be reduced to a linear separation (classification) problem.

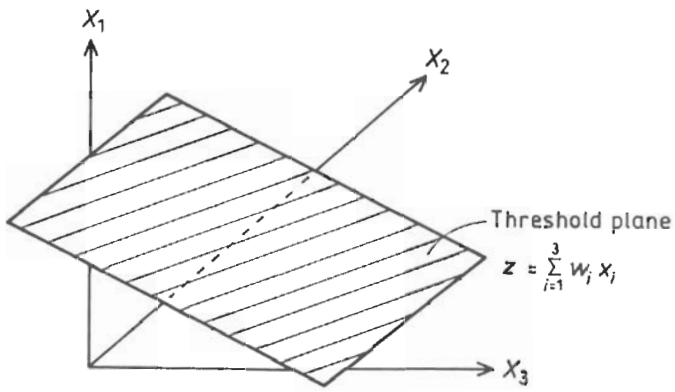


Fig. 4.7. A single layer's 3-input representation.

4.3. The Limitations of the Single-Layer Perceptron

In 1969, Minsky and Papert published a book (Minsky and Papert, 1969) where they pointed out as did E. B. Crane in 1965 in a less-known book, to the grave limitations in the capabilities of the perceptron, as is evident by its representation

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

theorem. They have shown that, for example, the perceptron cannot solve even a 2-state Exclusive-Or (XOR) problem $[(x_1 \cup x_2) \cap (\bar{x}_1 \cup \bar{x}_2)]$, as illustrated in the Truth-Table of Table 4.1, or its complement, the 2-state contradiction problem (XNOR).

Table 4.1. XOR Truth-Table.

	inputs		output
state	x_1	x_2	z
A	0	0	0
B	1	0	1
C	0	1	1
D	1	1	0

$(x_1$ or $x_2)$ and $(\bar{x}_1$ or $\bar{x}_2)$;
$\bar{x}$ denoting: not (x)

Table 4.2. Number of linearly separable binary problem. (based on P. P. Wasserman: Neural Computing Theory and Practice © 1989 International Thomson Computer Press. Reprinted with permission).

No. of inputs n	2^{2^n}	No. of linearly separable problems
1	4	4
2	16	14 (all but XOR, XNOR)
3	256	104
4	65 K	1.9 K
5	4.3×10^9	95 K
.	.	.
.	.	.
.	.	.
$n > 7$	x	$< x^{1/3}$

Obviously, no linear separation as in Fig. 4.1 can represent (classify) this problem.

Indeed, there is a large class of problems that single-layer classifiers cannot solve. So much so, that for a single layer neural network with an increasing number of inputs, the number of problems that can be classified becomes a very small fraction of the totality of problems that can be formulated.

Specifically, a neuron with binary inputs can have 2^n different input patterns. Since each input pattern can produce 2 different binary outputs, then there are 2^{2^n} different functions of n variables. The number of linearly separable problems of n binary inputs is however a small fraction of 2^{2^n} as is evident from Table 4.2 that is due to Windner (1960). See also Wasserman (1989).

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

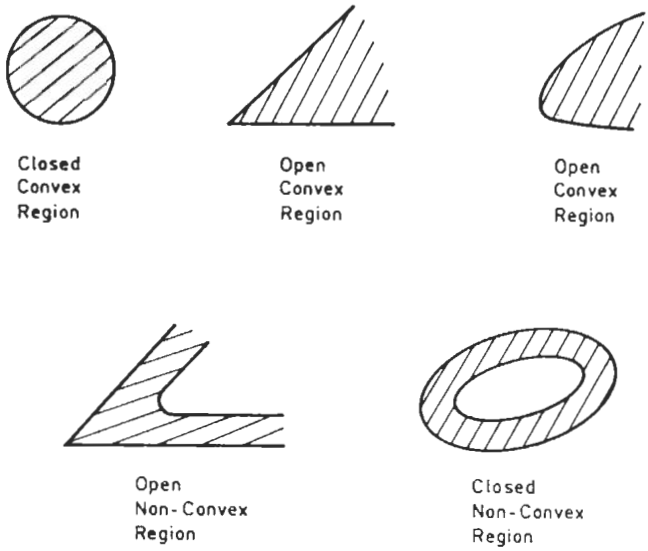


Fig. 4.8. Convex and non-convex regions.

4.4. Many-Layer Perceptrons

To overcome the limitations pointed out by Minsky and Papert, which at the time resulted in a great disappointment with ANNs and in a sharp drop (nearly total) of research into them, it was necessary to go beyond the single layer ANN.

Minsky and Papert (1969) have shown that a single-layer ANN can solve (represent) problems of classification of points that lie in a convex open region or in a convex closed region as in Fig. 4.8. (A convex region is one where any two points in that region can be connected by a straight line that lies fully in that region). In 1969 there was no method to set weights other than for neurons whose output (y) was accessible. It was subsequently shown [Rumelhart *et al.*, 1986] that a 2-layer ANN can solve also non-convex problems, including the XOR problem above. Extension to three or more layers extends the classes of problems that can be represented and hence solved by ANN to, essentially, no bound. However, in the 1960s and 1970s there was no powerful tool to set weights of a multi-layer ANN. Although multilayer training was already used to some extent for the Madaline, it was slow and not rigorous enough for the general multi-layer problem. The solution awaited the formulation of the Back Propagation algorithm, to be described in Chap. 6.

Our comments above, concerning a multi-layer Perceptron network, fully apply to any neuronal model and therefore to any multi-layer neural network, including all networks discussed in later chapters of this text. It therefore applies the Madaline of the next chapter and to recurrent networks (Chap. 7) whose recurrent structure makes a single layer behave as a dynamic multi-layer network.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

4.A. Perceptron Case Study: Identifying Autoregressive Parameters of a Signal (AR Time Series Identification)

Goal:

To model a time series parameter identification of a 5th order autoregressive (AR) model using a single Perceptron.

Problem Set Up:

First, a time series signal $x(n)$ of 2000 samples is generated using a *5th order AR model* added with white Gaussian noise $w(n)$. The mathematical model is as follows,

$$x(n) = \sum_{i=1}^{M} a_i x(n-i) + w(n) \tag{4.A.1}$$

where

$$M = \text{order of the model}$$

$$a_i = i\text{th element of the AR parameter vector } \alpha \text{ (alpha)}$$

The true AR parameters as have been used unknown to the neural network to generate the signal $x(u)$, are:

$$\begin{aligned} a_1 &= 1.15 \\ a_2 &= 0.17 \\ a_3 &= -0.34 \\ a_4 &= -0.01 \\ a_5 &= 0.01 \end{aligned}$$

The algorithm presented here is based on deterministic training. A stochastic version of the same algorithm and for the same problem is given in Sec. 11.B below. Given a time series signal $x(n)$, and the order M of the AR model of that signal, we have that

$$\hat{x}(n) = \sum_{i=1}^{M} \hat{a}_i x(n-i) \tag{4.A.2}$$

where $\hat{x}(n)$ is the estimate of $x(n)$, and then define

$$e(n) \triangleq x(n) - \hat{x}(n) \tag{4.A.3}$$

Therefore, if and when $\hat{a}_i$ have converged to a:

$$e(n) \to w(n) \tag{4.A.4}$$

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

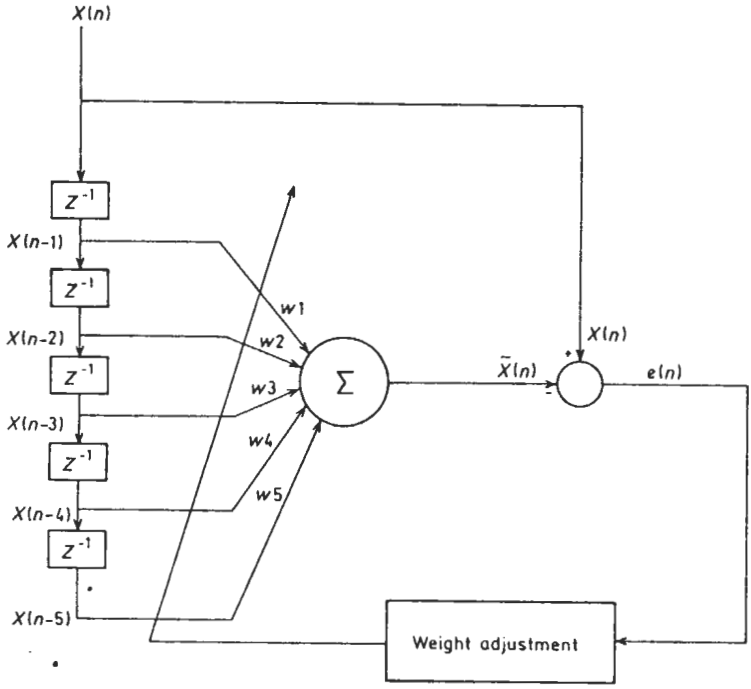


Fig. 4.A.1. Signal flow diagram.

The Perceptron neural network for this model is given in Fig. 4.A.1. Since the white Gaussian noise is uncorrelated with its past,

$$E[w(n)w(n-k)] = \begin{cases} \sigma_x^2 & \text{for } k = 0 \\ 0 & \text{otherwise} \end{cases} \tag{4.A.5}$$

Thus we define a mean square error (MSE) as

$$MSE \triangleq \hat{E}[e^2(n)] = \frac{1}{N}\sum_{i=1}^{N} e^2(i) \tag{4.A.6}$$

which is the sampled variance of the error $e(h)$ above over N samples

Deterministic Training:

Given $x(n)$ from Eq. (4.A.2), find $\hat{a}_i$ such that

$$\hat{x}(n) = \sum_{i=1}^{M} \hat{a}_i x(n-i) = \hat{a}^T x(n-1)$$

$$\hat{\mathbf{a}} \triangleq [\hat{a}_1 \cdots \hat{a}_M]^T$$

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

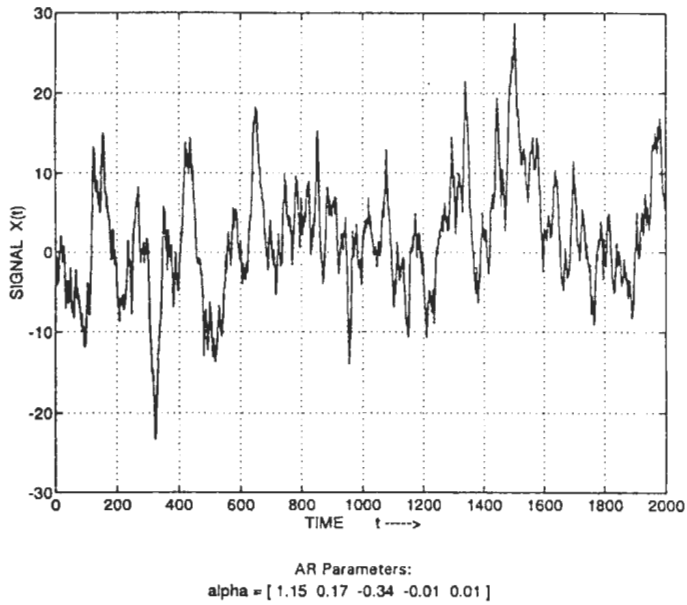


Fig. 4.A.2. Signal versus time.

then calculate

$$e(n) = x(n) - \hat{x}(n) \tag{4.A.7}$$

update the weight vector $\hat{\mathbf{a}}$ to minimize the MSE error of Eq. (4.A.6), by using the delta rule and momentum term

$$\Delta\hat{\mathbf{a}}(n) = 2\mu e(n)\mathbf{x}(n-1) + \alpha\Delta\hat{\mathbf{a}}(n-1) \tag{4.A.8}$$

$$\hat{\mathbf{a}}(n+1) = \hat{\mathbf{a}}(n) + \Delta\hat{\mathbf{a}}(n) \tag{4.A.9}$$

where

$$\hat{\mathbf{a}}(n) = [\hat{a}_1(n), \ldots, \hat{a}_5(n)]^T$$

$$\mathbf{x}(n-1) = [x(n-1) \cdots x(n-5)]^T$$

$$\mu_0 = 0.001$$

$$\alpha = 0.5$$

and μ is decreasing in iteration step as,

$$\mu = \frac{\mu_0}{1+k} \tag{4.A.10}$$

Note that α is a momentum coefficient which is added to the update equation since it can serve to increase the speed of convergence.

A plot of MSE versus the number of iteration is shown in Fig. 4.A.3. The flow chart of deterministic training is shown in Fig. 4.A.4.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

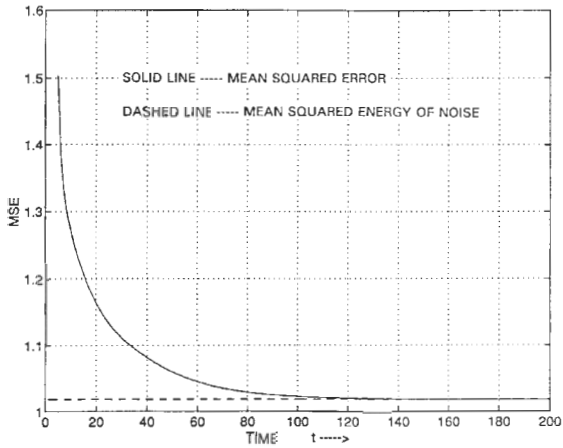


Fig. 4.A.3. Mean squared error versus time.

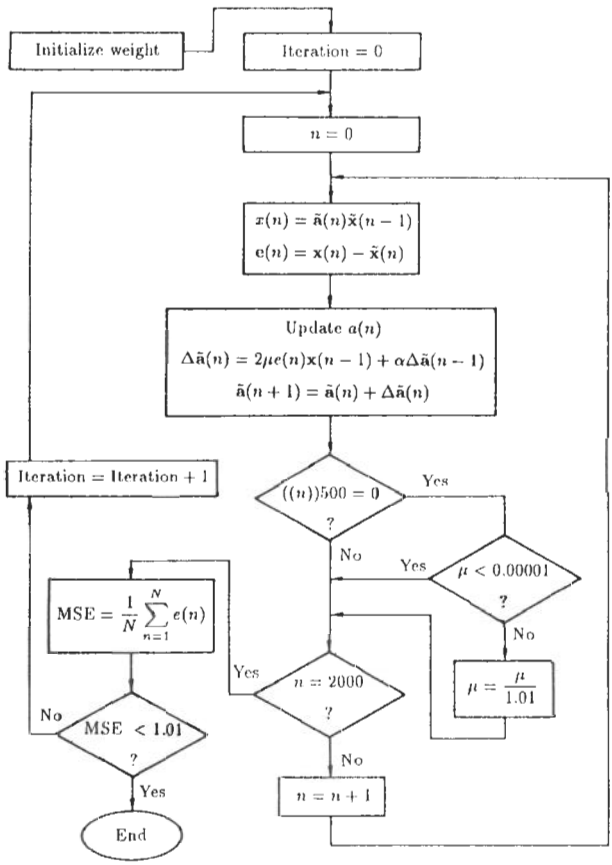


Fig. 4.A.4. Flow chart of deterministic training.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

Source CodeProgram — written in MATLAB® (MATLAB is a registered trademark of The MathWorks, Inc.)

```
%%%
```

Computational Results: Parameter Estimates (Weights) and Mean Square Error — Deterministic Training, No Bias Term Added

```
        1.15    0.17   -0.34   -0.01    0.01 = true (unknown) parameters

        0.598   0.446  -0.171   0.380  -0.231
1            MEANSQUARE ERROR = 117.0848

        0.743   0.353  -0.254   0.302  -0.131
2            MEANSQUARE ERROR = 17.5898

        0.808   0.300  -0.247   0.209  -0.062
3            MEANSQUARE ERROR = 4.2322

        0.838   0.271  -0.229   0.145  -0.021
4            MEANSQUARE ERROR = 1.9580

        0.856   0.253  -0.214   0.106   0.002
5            MEANSQUARE ERROR = 1.5033

        0.869   0.240  -0.205   0.082   0.014
6            MEANSQUARE ERROR = 1.3857

        0.881   0.230  -0.201   0.069   0.021
7            MEANSQUARE ERROR = 1.3386

        0.892   0.221  -0.200   0.060   0.026
8            MEANSQUARE ERROR = 1.3103

        0.901   0.214  -0.201   0.054   0.029
9            MEANSQUARE ERROR = 1.2892

        0.911   0.208  -0.203   0.050   0.031
10           MEANSQUARE ERROR = 1.2717

        0.920   0.203  -0.206   0.046   0.032
11           MEANSQUARE ERROR = 1.2564

        0.928   0.199  -0.209   0.042   0.034
12           MEANSQUARE ERROR = 1.2427

        0.936   0.195  -0.212   0.039   0.035
13           MEANSQUARE ERROR = 1.2301

        0.944   0.191  -0.215   0.036   0.036
14           MEANSQUARE ERROR = 1.2185

        0.951   0.188  -0.218   0.034   0.036
15           MEANSQUARE ERROR = 1.2077
```

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

```
32          MEANSQUARE ERROR = 1.1047

       1.040   0.165  -0.262   0.000   0.036
33          MEANSQUARE ERROR = 1.1014

       1.043   0.165  -0.264  -0.001   0.035
34          MEANSQUARE ERROR = 1.0982

       1.046   0.164  -0.266  -0.003   0.035
35          MEANSQUARE ERROR = 1.0952

       1.050   0.164  -0.268  -0.004   0.035
36          MEANSQUARE ERROR = 1.0923

       1.053   0.163  -0.269  -0.006   0.034
37          MEANSQUARE ERROR = 1.0894

       1.056   0.163  -0.271  -0.007   0.034
38          MEANSQUARE ERROR = 1.0867

       1.059   0.163  -0.273  -0.008   0.033
39          MEANSQUARE ERROR = 1.0840

       1.062   0.162  -0.274  -0.009   0.033
40          MEANSQUARE ERROR = 1.0814

       1.064   0.162  -0.276  -0.011   0.033
41          MEANSQUARE ERROR = 1.0789

       1.067   0.162  -0.277  -0.012   0.032
42          MEANSQUARE ERROR = 1.0765

       1.070   0.162  -0.278  -0.013   0.032
43          MEANSQUARE ERROR = 1.0741

       1.072   0.161  -0.280  -0.014   0.031
44          MEANSQUARE ERROR = 1.0718

       1.075   0.161  -0.281  -0.015   0.031
45          MEANSQUARE ERROR = 1.0696

       1.077   0.161  -0.282  -0.016   0.030
46          MEANSQUARE ERROR = 1.0675

       1.079   0.161  -0.283  -0.018   0.030
47          MEANSQUARE ERROR = 1.0654

       1.081   0.161  -0.284  -0.019   0.030
48          MEANSQUARE ERROR = 1.0634
```

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

```
        1.137   0.169  -0.302  -0.040   0.019
181          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.019
182          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.019
183          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.019
184          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.019
185          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.019
186          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.019
187          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.019
188          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.019
189          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.020
190          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.020
191          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.020
192          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.020
193          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.020
194          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.020
195          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.020
196          MEANSQUARE ERROR = 1.0181

        1.137   0.169  -0.302  -0.040   0.020
```

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

```
197          MEANSQUARE ERROR = 1.0181

       1.137   0.169  -0.302  -0.040   0.020
198          MEANSQUARE ERROR = 1.0181

       1.137   0.169  -0.302  -0.040   0.020
199          MEANSQUARE ERROR = 1.0181

       1.137   0.169  -0.302  -0.040   0.020
200          MEANSQUARE ERROR = 1.0181
```

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

Parameter Estimates (Weights) and The Mean Square Error Deterministic Training only with Bias Term Added

```
        1.15    0.17  -0.34   -0.01    0.01 = true (unknown) parameters

        0.587   0.451  -0.180   0.380  -0.246
 1           MEANSQUARE ERROR = 122.0708

        0.733   0.353  -0.257   0.303  -0.147
 2           MEANSQUARE ERROR = 18.3725

        0.800   0.297  -0.249   0.209  -0.076
 3           MEANSQUARE ERROR = 4.3699

        0.831   0.268  -0.230   0.145  -0.035
 4           MEANSQUARE ERROR = 1.9879

        0.850   0.249  -0.215   0.105  -0.011
 5           MEANSQUARE ERROR = 1.5141

        0.863   0.236  -0.205   0.081   0.002
 6           MEANSQUARE ERROR = 1.3927

        0.875   0.226  -0.201   0.067   0.009
 7           MEANSQUARE ERROR = 1.3445

        0.886   0.217  -0.200   0.058   0.014
 8           MEANSQUARE ERROR = 1.3158

        0.896   0.210  -0.201   0.052   0.017
 9           MEANSQUARE ERROR = 1.2943

        0.906   0.204  -0.203   0.048   0.020
 10          MEANSQUARE ERROR = 1.2765

        0.915   0.199  -0.206   0.044   0.022
 11          MEANSQUARE ERROR = 1.2609

        0.924   0.195  -0.209   0.040   0.023
 12          MEANSQUARE ERROR = 1.2469

        0.932   0.191  -0.212   0.037   0.025
 13          MEANSQUARE ERROR = 1.2341

        0.940   0.188  -0.215   0.035   0.026
 14          MEANSQUARE ERROR = 1.2223

        0.947   0.185  -0.218   0.032   0.027
 15          MEANSQUARE ERROR = 1.2113
```

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

```
        1.136   0.169  -0.301  -0.040   0.018
181          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.301  -0.040   0.018
182          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.301  -0.040   0.018
183          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.301  -0.040   0.018
184          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.301  -0.040   0.018
185          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.301  -0.040   0.018
186          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.301  -0.040   0.018
187          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.301  -0.040   0.018
188          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.301  -0.040   0.018
189          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.301  -0.040   0.018
190          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.302  -0.040   0.018
191          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.302  -0.040   0.018
192          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.302  -0.040   0.018
193          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.302  -0.040   0.018
194          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.302  -0.040   0.018
195          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.302  -0.040   0.018
196          MEANSQUARE ERROR = 1.0168
```

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

```
        1.136   0.169  -0.302  -0.040   0.018
197          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.302  -0.040   0.018
198          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.302  -0.040   0.018
199          MEANSQUARE ERROR = 1.0168

        1.136   0.169  -0.302  -0.040   0.018
200          MEANSQUARE ERROR = 1.0168
```

Observe the closeness of the parameters identified above (say, at iteration 200) to the original but unknown parameters as at the beginning of Sec. 4.A.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/26/2026 4:59:31 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.